

Combinatorial Optimization

Optimization problems, and the combinatorial family

Lecture 1

Term 261

Optimization Problems

- An optimization problem is about finding the best solution among many possible choices.
- Formal idea:
 - Objective: minimize or maximize a function (e.g., cost, time, profit).
 - Subject to: certain rules or constraints.
- Examples:
 - Finding the shortest route to work
 - Choosing items to pack in a bag to maximize value without exceeding weight

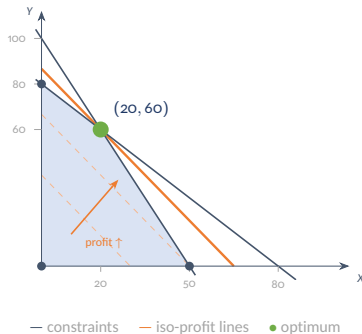
Types of Optimization Problems

- 1. Linear Optimization (Linear Programming)
 - The objective and constraints are linear equations/inequalities.
 - Example: A company makes two products, X and Y.
 - Profit = $40X + 30Y$ (maximize)
 - Constraints:
 - $2X + Y \leq 100$ (resource 1 limit)
 - $X + Y \leq 80$ (resource 2 limit)
 - $X, Y \geq 0$

Linear example: the search space

The feasible set is an intersection of half-planes: a **convex polygon**. The iso-profit lines $40X + 30Y = c$ are parallel, so raising c slides them outward until they last touch the polygon — and that last contact is a corner.

Vertex	Profit
(0, 0)	0
(50, 0)	2000
(0, 80)	2400
(20, 60)	2600



At $(20, 60)$ both constraints are tight — every unit of both resources is consumed.

Takeaway: With a linear objective over a polygon the optimum always sits at a *vertex*. That is why simplex only ever walks corner to corner, and why finitely many candidates suffice.

Types of Optimization Problems

- 2. Non-Linear Optimization
 - At least one part (objective or constraints) is non-linear.
 - Example: Maximize profit for two products, X and Y, where production cost grows quadratically.
 - Profit = $50X + 40Y - (0.5X^2 + 0.25Y^2)$ (maximize)
 - Constraints:
 - $X + 2Y \leq 100$
 - $X, Y \geq 0$

Non-linear example: the search space

1. **Ignore the constraint.** $\partial_X f = 50 - X$ and $\partial_Y f = 40 - \frac{Y}{2}$ vanish at $(50, 80)$, worth 2850 — but $50 + 2(80) = 210 > 100$, so it is infeasible and the constraint must be active.

2. **Optimize along $X + 2Y = 100$.** Substituting $X = 100 - 2Y$,

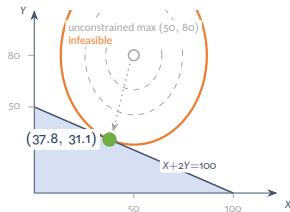
$$f(Y) = 140Y - 2.25Y^2, \quad f'(Y) = 140 - 4.5Y = 0.$$

Optimum

$$X^* = \frac{340}{9} \approx 37.8, \quad Y^* = \frac{280}{9} \approx 31.1, \\ f^* = \frac{19600}{9} \approx 2177.8$$

KKT check: $\lambda = 50 - X^* = \frac{110}{9}$ and $2\lambda = 40 - \frac{Y^*}{2} = \frac{220}{9}$ agree, with $\lambda > 0$ as a binding $\leq$ constraint requires.

Takeaway: The contours are curved, so the optimum is a *tangency* on an edge, not a corner. Enumerating vertices finds nothing here — which is precisely what separates non-linear from linear programming.



— contour $f = 2177.8$ - - lower contours • optimum

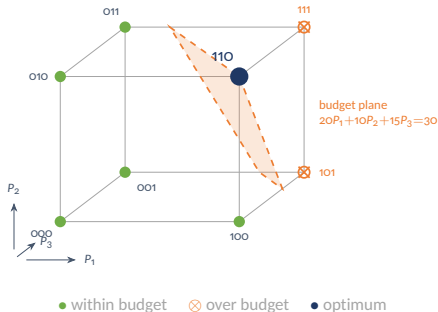
Types of Optimization Problems

- 3. Integer Optimization (Integer Programming)
 - Decision variables must be integers (often 0/1 for yes/no decisions).
 - Example: A company must decide which projects to fund (0 = not funded, 1 = funded).
 - Maximize total profit = $60P_1 + 40P_2 + 30P_3$
 - Constraints:
 - $20P_1 + 10P_2 + 15P_3 \leq 30$ (budget limit)
 - $P_1, P_2, P_3 \in \{0,1\}$

Integer example: the search space

Three binary variables give $2^3 = 8$ candidate portfolios — few enough to enumerate.

(P_1, P_2, P_3)	Cost	Profit	
(0, 0, 0)	0	0	
(0, 0, 1)	15	30	
(0, 1, 0)	10	40	
(1, 0, 0)	20	60	
(0, 1, 1)	25	70	
(1, 1, 0)	30	100	optimum
(1, 0, 1)	35	90	over
(1, 1, 1)	45	130	over



Fund projects 1 and 2: profit 100, spending the budget exactly.

Takeaway: The search space is a *finite set of corners*, not a region — there is no interior to slide along. The cube is the LP relaxation; relax, solve, round is the standard attack, and it is exactly what breaks on the problems in the rest of this course.

Combinatorial Optimization

- A class of optimization problems where the solution must be chosen from a discrete set of possibilities (often finite but huge in number).
- Goal: Find the best arrangement, selection, or structure.
- Examples:
 - Travelling Salesman Problem (shortest route visiting all cities).
 - Knapsack Problem (choose items to maximize value under capacity).
 - Graph Coloring (assign colors to vertices so no two neighbors share the same color).

Types of Combinatorial Optimization Problems

- Routing Problems: About finding the best paths, tours, or schedules.
- Assignment Problems: Assign tasks/resources to agents in the best way.
- Knapsack-type Problems: Selecting items under constraints (weight, capacity, cost) to maximize value.
- Network Design Problems: Choosing or structuring connections in a network.
- Partitioning & Clustering Problems: Dividing a set into groups to optimize some objective.
- Scheduling Problems: Allocate resources over time efficiently.

Traveling Salesman Problem (TSP)

- Suppose a salesman needs to visit 4 cities (A, B, C, D), starting and ending at A
- The distance between each pair of cities is given in the table below
- Goal: find the shortest possible tour that visits each city exactly once and returns to A

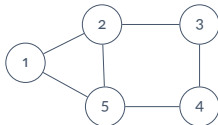
	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Knapsack Problem

- Given
 - A knapsack with limited capacity
 - A set of items each with a capacity and a value
- Goal: choose a subset of items to put into the knapsack so that
 - The total weight does not exceed the capacity
 - The total value is as large as possible.
- Example:
 - Knapsack capacity = 10
 - Items:
 - Item 1: weight 6, value 30
 - Item 2: weight 3, value 14
 - Item 3: weight 4, value 16

Max-cut Problem

- Given a graph with nodes (vertices) and edges (which may have weights)
- The task is to divide the nodes into two groups (a partition)
- Goal: maximize the total weight (or number) of edges that cross between the two groups



All six edges have weight 1, so the cut value is simply the number of crossing edges.

Example. Five vertices, six edges, every weight 1:

$$V = \{1, 2, 3, 4, 5\}, \quad E = \{12, 15, 23, 25, 34, 45\}.$$

Split V into S and $S' = V \setminus S$. An edge is *cut* when its two endpoints land on opposite sides.

▮ *Six edges. How many of them can be made to cross at once — and can you get all six?*

Max-cut example: the search space

Each vertex goes left or right, so there are $2^5 = 32$ assignments — but swapping the two sides gives the same cut, leaving **16 distinct partitions**. Small enough to enumerate:

Cut value	Partitions
5	2
4	3
3	6
2	4
0	1
total	16

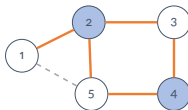
Optimum: cut = 5

$$S = \{2, 4\}, \quad S' = \{1, 3, 5\}$$

crossing: 12, 23, 25, 34, 45 uncut: 15

$S = \{3, 5\}$ also gives 5, leaving 12 uncut — optima need not be unique.

Takeaway: A graph is fully cuttable exactly when it is *bipartite* — no odd cycles. This graph has one triangle, so 5 is optimal, not merely the best found.



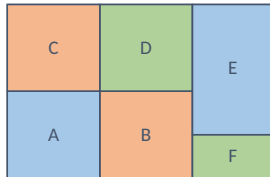
● S ○ S' — cut — uncut

Why not all six? Vertices 1, 2, 5 form a **triangle**. Two sides cannot hold three mutually adjacent vertices without two of them sharing a side, so at least one triangle edge never crosses:

$$\text{max-cut} \leq |E| - 1 = 5.$$

Graph Coloring Problem

- Graph coloring is the problem of assigning colors to the vertices of a graph so that:
 - No two adjacent vertices share the same color.
 - The goal is to use the minimum number of colors
 - Example: map coloring puzzle



Three colours suffice here.

Assignment Problem

- The Assignment Problem is about assigning tasks or jobs to agents (e.g., workers, machines) so that each agent gets exactly one task and the total cost is minimized (or total profit is maximized)
- Example: 3 workers (W1, W2, W3) and 3 jobs (J1, J2, J3).
- Cost matrix:
- Objective: Assign each worker to a unique job to minimize total cost.
- Optimal assignment: $W1 \rightarrow J2$, $W2 \rightarrow J3$, $W3 \rightarrow J1$, with total cost $2 + 3 + 4 = 9$

	J1	J2	J3
W1	9	2	7
W2	6	4	3
W3	5	8	1

Scheduling Problem

- A Scheduling Problem is about allocating tasks to resources (e.g., machines, workers, time slots) over time, aiming to minimize total time, cost, or lateness
- Example:
- One machine and 3 jobs with processing times: $J_1 = 2$, $J_2 = 4$, $J_3 = 3$ If order is $J_1 \rightarrow J_3 \rightarrow J_2$
- Completion times = 2, 5, 9 Total = 16 (optimal)
- Objective: Find job order that minimizes total completion time.

Takeaway Questions

- All optimization problems share key elements:
 - Objective function: a quantity to minimize or maximize (e.g., cost, time, profit)
 - Decision variables: represent the choices to be made (e.g., how many units, yes/no, order of tasks)
 - Constraints: rules or limits that must be respected (e.g., budget, resources, capacity)
 - Feasible solutions: all choices that satisfy the constraints
 - Optimal solution: the feasible solution that gives the best objective value
 - Easy to validate: if the solution is magically available, it's easy to check optimality

Next

- Modeling combinatorial optimization problems
- Hardness of combinatorial optimization problems
- Algorithms and techniques and their optimality
- Equivalency of optimization problems
- Variants of combinatorial optimization problems